```
   "outputs": [],
   "source": [
    "def entropy(p):\n",
    "    return -np.sum(p * np.log(p + 1e-12), axis=1)\n",
    "\n",
    "rng = np.random.RandomState(0)\n",
    "members = []\n",
    "for i in range(7):\n",
    "    idx = rng.choice(len(X_train), len(X_train), replace=True)\n",
    "    m = MLPClassifier(hidden_layer_sizes=(64,), max_iter=500, random_state=i +
1)\n",
    "    m.fit(X_train[idx], y_train[idx])\n",
    "    members.append(m)\n",
    "\n",
    "def guardian_uncertainty(data):\n",
    "    P = np.stack([m.predict_proba(data) for m in members])   # (مدل نمونه کلاس)\n",
    "    mean_p = P.mean(axis=0)                                  # رأی میانگین هفت
مدل \n",
    "    return entropy(mean_p)                                   # تردید جمعی\n",
    "\n",
    "print(\"نگهبان ساخته شد\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
     ۴                                             مقایسهٔ دو روش با عدد\" ## گام
    "- **AUROC**: ۱                                 هرچه به
     ۹۵                  ت رد شوند، و می‌شماریم- **درصد گرفتن ناشناخته‌ها**: آستانه را طوری می‌گذاریم که
                                                   "ها گرفته شهچند درصد ناشناخته
   ]
  },
  {
   "cell_type": "code",
   "metadata": {},
   "execution_count": null,
   "outputs": [],
   "source": [
    "def evaluate(unc_fn, name):\n",
    "    u_known = unc_fn(X_test)\n",
    "    u_unknown = unc_fn(X_unknown)\n",
    "    labels = np.r_[np.zeros(len(u_known)), np.ones(len(u_unknown))]\n",
    "    scores = np.r_[u_known, u_unknown]\n",
    "    auroc = roc_auc_score(labels, scores)\n",
    "    threshold = np.percentile(u_known, 95)          # آشناها زیر این حد ۹۵٪ \n",
    "    caught = np.mean(u_unknown > threshold)\n",
    "    print(\"%-22s AUROC = %.3f |                                        ناشناخته
caught))\n",
    "    return u_known, u_unknown, threshold, auroc, caught\n",
    "\n",
    "res_base = evaluate(base_uncertainty, \"مدل ساده\")\n",
    "res_guard = evaluate(guardian_uncertainty, \"نگهبان لوگوس\")"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {},
   "execution_count": null,
   "outputs": [],
   "source": [
    "fig, axes = plt.subplots(1, 2, figsize=(12, 4))\n",
    "for ax, res, title in zip(axes, [res_base, res_guard],\n",
    "                           [\"Simple model\", \"LOGOS guardian\"]):\n",
    "    u_known, u_unknown, thr, auroc, caught = res\n",
    "    ax.hist(u_known, bins=30, alpha=0.6, label=\"Known digits (0-6)\",
density=True)\n",
    "    ax.hist(u_unknown, bins=30, alpha=0.6, label=\"Unknown digits (7-9)\",
density=True)\n",
    "    ax.axvline(thr, linestyle=\"--\", color=\"k\", label=\"Alarm threshold\")\n",
    "    ax.set_title(\"%s\\nAUROC=%.3f, caught %.0f%% of unknowns\" % (title, auroc,
100 * caught))\n",
    "    ax.set_xlabel(\"Uncertainty\")\n",
    "    ax.legend()\n",
    "plt.tight_layout()\n",
    "plt.savefig(\"logos_result.png\", dpi=150)\n",
    "plt.show()"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    ۴                                                            خواندن صادقانهٔ نتیجه‌ها\"## گام
    ۵                                داده‌ٔ آموزش و تست تکرار "یک اجرای تکی ممکن است شانسی باشد. پس همین مقایسه را روی
                                                                         می‌کنیم\n",
    "                                                                     و میانگین می
   ]
  },
  {
   "cell_type": "code",
   "metadata": {},
   "execution_count": null,
   "outputs": [],
   "source": [
    "def run_once(seed):\n",
    "    Xtr, Xte, ytr, yte = train_test_split(\n",
    "        X_known, y_known, test_size=0.3, random_state=seed, stratify=y_known)\n",
    "    b = MLPClassifier(hidden_layer_sizes=(64,), max_iter=500,
random_state=0).fit(Xtr, ytr)\n",
    "    r = np.random.RandomState(seed)\n",
    "    ms = []\n",
    "    for i in range(7):\n",
    "        idx = r.choice(len(Xtr), len(Xtr), replace=True)\n",
    "        ms.append(MLPClassifier(hidden_layer_sizes=(64,), max_iter=500,\n",
    "                                random_state=i + 1).fit(Xtr[idx], ytr[idx]))\n",
    "    ub = lambda D: 1 - b.predict_proba(D).max(axis=1)\n",
    "    ug = lambda D: entropy(np.mean([m.predict_proba(D) for m in ms], axis=0))\n",
    "    labels = np.r_[np.zeros(len(Xte)), np.ones(len(X_unknown))]\n",
    "    return (roc_auc_score(labels, np.r_[ub(Xte), ub(X_unknown)]),\n",
    "            roc_auc_score(labels, np.r_[ug(Xte), ug(X_unknown)]))\n",
    "\n",
    "runs = np.array([run_once(s) for s in range(5)])\n",
    "print(\"AUROC  3                                          %. میانگین  :  مدل ساده f\" % runs[:, 0].m
3))\n",
    "print(\"AUROC  3                                          %. میانگین  :    نگهبان f\" % runs[:, 1].m
3))\n",
    "diff = runs[:, 1].mean() - runs[:, 0].mean()\n",
    "print()\n",
    "if diff > 0.01:\n",
    "    print(\"                                                       نتیجه3: نگهبان به
    "else:\n",
    "    print(\" 3                     ام کار آسانی . (fنتیجه: در این مسئلهٔ ساده تفاوت کوچک است (%+.
.\" % diff)\n",
    "    print(\"                                                      ناشناخته          مهم
                                                       کربلا اشتباه می.\")\n",
    "    print(\"                                  ن باید خودش را ثابت کگام بعد: مسئلهٔ سخت
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    ۵                                                    گام ##"\لوگوس «گفتار» :n",
    \ اولیهٔ گفتار یعنی توضیح تصمیم به زبان انسان. این نسخهٔ اولیه ی‌ک گزارش متنی ساده می‌دهد.n",
    "                                                    بعداً می
   ]
  },
  {
   "cell_type": "code",
   "metadata": {},
   "execution_count": null,
   "outputs": [],
   "source": [
    "threshold = res_guard[2]\n",
    "\n",
    "def logos_report(sample):\n",
    "    sample = sample.reshape(1, -1)\n",
    "    u = guardian_uncertainty(sample)[0]\n",
    "    P = np.stack([m.predict_proba(sample)[0] for m in members])\n",
    "    guess = int(np.argmax(P.mean(axis=0)))\n",
    "    if u > threshold:\n",
    "        return (\"⚠                                     هشدار لوگوس: مدل
%.3f). \"\n",
    "                  \"این وضعیت شبیه چیزی نیست که یاد گرفته‌ام. پیشنهاد: کاهش سرعت و تأیید انسان.\"
% (u, threshold))\n",
    "    return \" حدس: رقم ( وضعیت آشنا %d، 3 .% عدم قطعیت f). ادامهٔ کار عادی.\" % (guess,
u)\n",
    "\n",
    "print(\"--- یک نمونهٔ آشنا ---\")\n",
    "print(logos_report(X_test[0]))\n",
    "print()\n",
    "print(\"--- چند نمونهٔ ناشناخته ---\")\n",
    "for i in range(5):\n",
    "    print(logos_report(X_unknown[i]))"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## قدم بعدی\n",
    ` ) اولین «شواهد» توسط هرچه عدد نشان داد همان"logos_result.png - نتیجهٔ این نوت‌بوک (عددها و نمودار `
بیشتر نه بنویس را\n",
    "- یافتهٔ محکم فعلی: مدل ساده روی چیز ناشناخته گاهی با اطمینان بالا اشتباه می‌کند. تفاوت نگهبان با مدل ساده را
باید در مسئلهٔ سخت‌تر اثبات کنیم.\n",
    \ قدم بعد: به‌جای ارقام از تصاویر جاده ( مثلاً مجموعه‌دادهٔ عمومی رانندگی) استفاده کنیم.n",
    "بعد از آن: لایهٔ «ناوگان» که اگر یک خودرو ناشناخته‌ی دید، بقیه را خبر کند -"
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "Python 3",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "name": "python"
  },
  "colab": {
   "provenance": []
  }
 },
 "nbformat": 4,
 "nbformat_minor": 5
}
```